

PLAN D'ACTION ÉCO-CONCEPTION À 6 MOIS

beer/me × heavy-ops

Made with GAMMA

HEAVY-OPS FOURNIT LE TERRAIN DE MESURE AVANT BEER/ME



beer/me : SaaS d'accompagnement du brasseur artisanal, en cours de développement



heavy-ops : dépôt-miroir utilisé comme terrain d'entraînement éco-conception

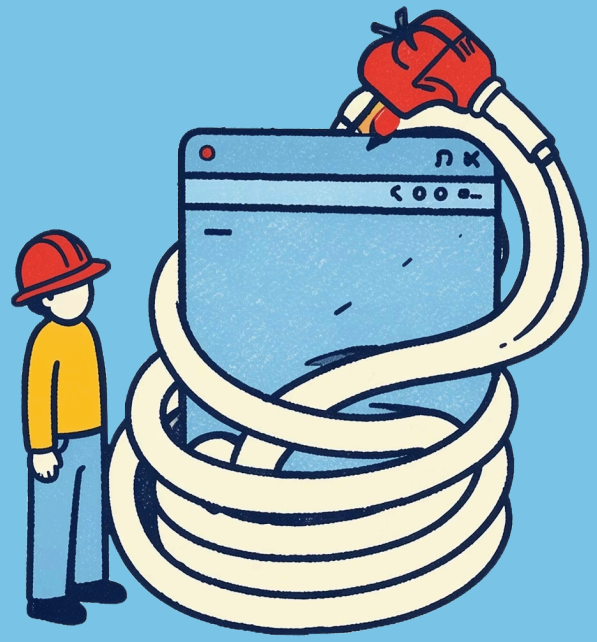


Objectif : réduire l'empreinte réseau, CPU et stockage sur 6 mois, puis transposer les règles vers beer/me

Made with GAMMA

MÉTHODE D'AUDIT

- **Outils :** extension GreenIt, Lighthouse, devtools Network/CPU — limités au chargement initial d'une page
- **Score EcoIndex** mesuré sur 4 écrans (accueil, table, analytics, settings)
- **Accueil :** 69.59 (note C) — le moins bon score, DOM le plus lourd (366 nœuds)
- **Table / Analytics / Settings :** 73-79 (note B)
- **Constat :** 23 requêtes par écran, ~2 Mo transférés sur un chargement unique — un score qui semble correct
- **Mais :** ces outils ne mesurent pas le coût d'usage dans le temps (polling) — complété par une mesure de session dédiée (slide suivante)



Made with GAMMA

LE COÛT CACHÉ : MESURE DE SESSION

≈ 152,6 KO

toutes les 5 secondes

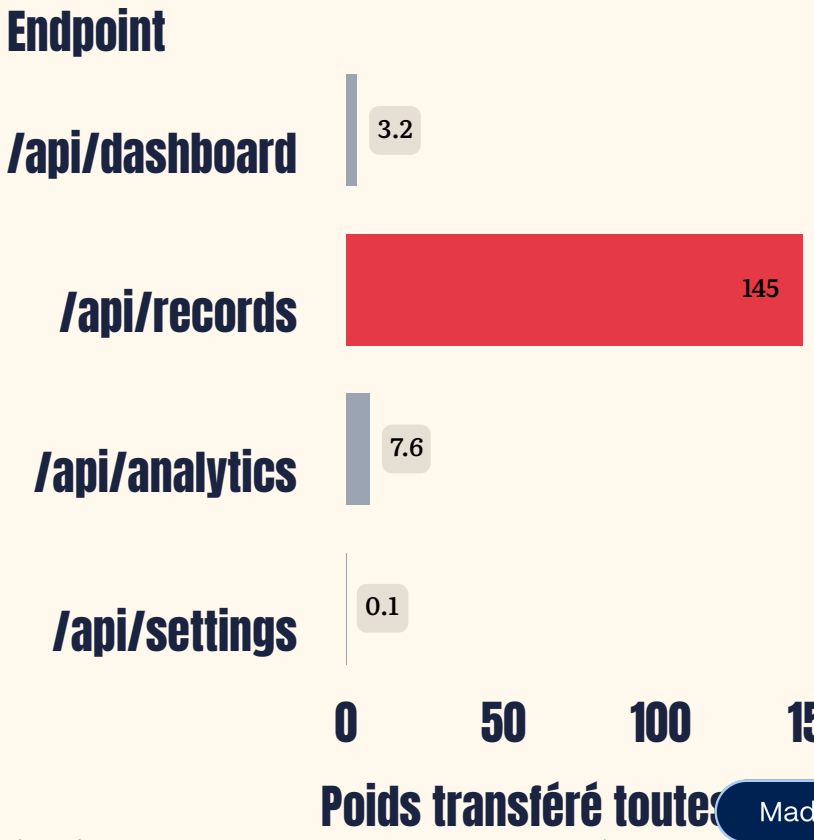
≈ 17,9 MO

en 10 minutes

95%

du volume via /api/records

- 4 endpoints interrogés toutes les 5 secondes (setInterval(loadAll, 5000), OpsApp.tsx)
- Poids mesuré par endpoint : dashboard 3,2 Ko / records 145,2 Ko / analytics 7,6 Ko / settings 0,1 Ko
- ≈ 152,6 Ko transférés toutes les 5 secondes, pour un onglet simplement ouvert
- Soit ≈ 17,9 Mo en 10 minutes (≈ 1,8 Mo/minute) — sans aucune interaction utilisateur
- /api/records représente à lui seul 95% de ce volume



/api/records = 95% du volume transféré (145,2 Ko sur ≈ 152,6 Ko)

Made with GAMMA

QUATRE ANTI-PATTERNS CONCENTRENT LES PRINCIPAUX GAINS

Polling :

setInterval(loadAll, 5000) sur 4 endpoints — effort faible, impact haut

Chargement intégral :

records.json / analytics.json sans pagination — effort moyen, impact haut

Calculs client :

teamSummary / urgentRecords recalculés à chaque render — effort moyen, impact moyen

Ressources :

SVG non référencés dans assets/ — effort très faible, impact faible

Made with GAMMA

DE LA MESURE À L'ACTION

Mesure de référence : ≈ 152,6 Ko toutes les 5 secondes, soit ≈ 17,9 Mo en 10 minutes pour un onglet ouvert

Règle beer/me : pas de polling par défaut, pagination serveur par défaut, calculs métier côté serveur par défaut.

1

Stopper le polling systématique

Rafraîchir sur action de l'utilisateur ou à intervalle justifié.

Gain attendu : moins de requêtes automatiques et de volume transféré.

2

Paginer /api/records

Ne charger que les données utiles à l'écran.

Gain attendu : moins de poids de réponse, qui pèse 95 % du volume mesuré.

3

Déplacer les calculs métier côté serveur

Exposer des données dérivées prêtes à afficher.

Gain attendu : moins de charge CPU et de recalculs côté client.

Made with GAMMA

LE BACKLOG CIBLE D'ABORD LES GAINS RÉSEAU ET CPU

US001 — haute :	réduire le polling réseau, objectif −80 % de requêtes automatiques
US002 — haute :	paginer /api/records, objectif −90 % de poids de réponse
US003 — moyenne :	déporter les calculs dérivés côté serveur
US004 — basse :	nettoyer les SVG non référencés
US005 — haute :	mettre en place un linter avant toute évolution de code
US006 — basse :	regrouper et dédupliquer les règles CSS
US007 — moyenne :	charger paresseusement les routes et composants lourds
US008 — basse :	mettre en cache les assets statiques avec un Service Worker
US009 — basse :	définir une stratégie de rétention et d'archivage des données

Made with GAMMA

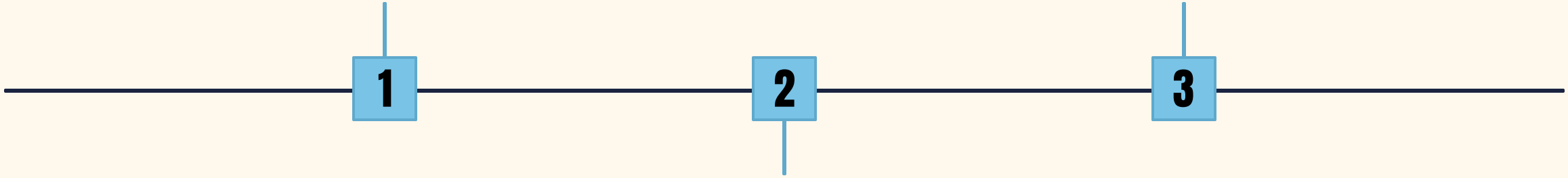
M1 À M3 : MESURER, RÉDUIRE, DÉPLACER

M1

US005 linter, US001 polling, US004 SVG — établir la baseline Network

M3

US003 calculs serveur — profiler React et comparer la charge CPU avant/après



M2

US002 pagination — viser −90 % sur le poids de réponse API

Made with GAMMA

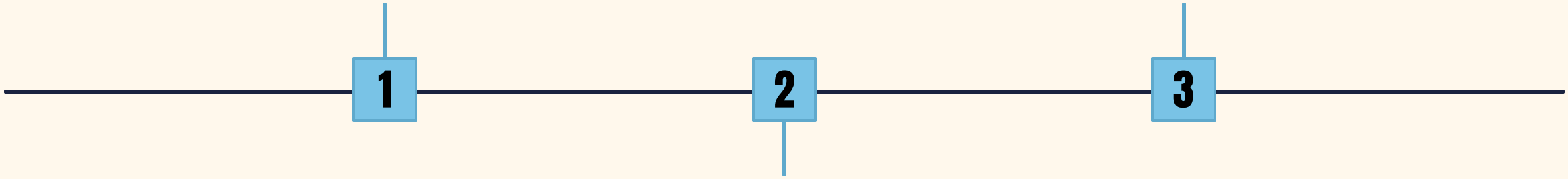
M4 À M6 : CONSOLIDER ET APPRENDRE

M4

US006 CSS et US007 lazy loading — suivre le poids CSS et le bundle JS

M6

US009 rétention des données, puis rétrospective finale



M5

US008 Service Worker — mesurer le cache hit ratio à la deuxième visite

Made with GAMMA

LES KPI RELIENT CHAQUE US À UN RÉSULTAT MESURABLE

Nombre de requêtes réseau par minute

Volume de données transféré sur 10 minutes

Poids de la réponse /api/records

Charge CPU et temps de rendu avec le profiler React

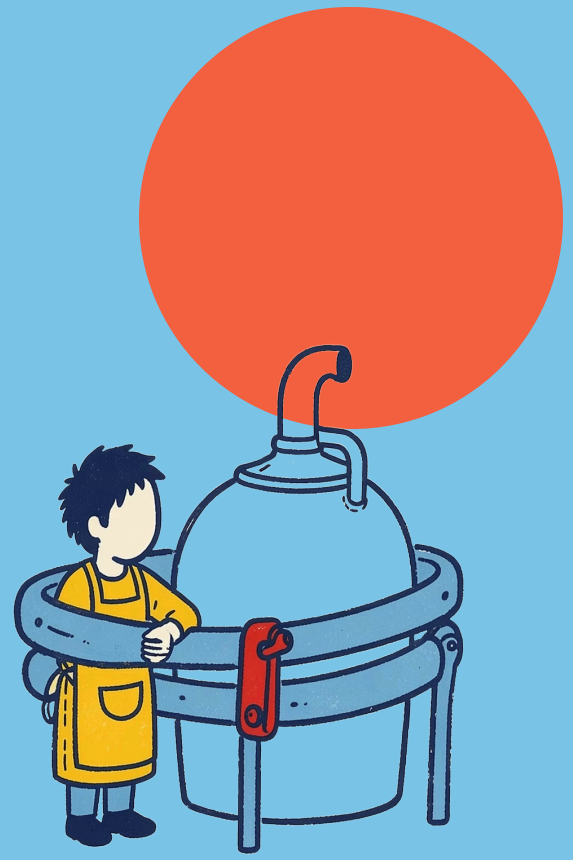
Poids du bundle JS et de ops.css

Volume stocké dans records.json et analytics.json

Made with GAMMA

BEER/ME DÉMARRE AVEC DES GARDE-FOUS, PAS AVEC DU RATTRAPAGE

- Pas de polling systématique ; pagination serveur par défaut ; calculs métier côté serveur
- Linter et CSS maîtrisés dès le premier commit
- Stratégie de rétention des données de brassin pensée dès le schéma de données
- **Prochaines étapes :** implémenter les US par branches dédiées, mesurer avant/après à chaque US, puis trancher les règles définitives en M6



Made with GAMMA